□ Lab 1.1 — Your First Python Program

□ Objective

By the end of this lab, you will have:

- Used the Python interactive shell (REPL) to run code instantly
- Written your first Python script file (`.py`)
- Run a Python program from the terminal
- Understood what an interpreted language means in practice

□ Background

What is Python?

Python is a **high-level, interpreted programming language** created by Guido van Rossum in 1991. It is designed for **readability** — Python code looks almost like plain English, which is why beginners pick it up quickly and professionals love it for automation tasks.

Interpreted vs. Compiled — What's the Difference?

This is a foundational concept in programming. Understanding it will help you understand how Python behaves.

	Interpreted (Python)	**Compiled (C, C++)**
How it works	Code is read and executed **line by line** at runtime	Code is translated entirely into machine code **before** running
Speed	Slightly slower at runtime	Faster at runtime
Development speed	Much faster — run code immediately	Slower — must compile before testing
Error discovery	Errors appear as the program runs	Errors appear at compile time
Examples	Python, JavaScript, Ruby	C, C++, Go, Rust

In practice for IT automation: Python's interpreted nature means you can test a one-liner instantly, modify scripts without recompiling, and run code on any machine that has Python installed — no build process required.

What is the REPL?

REPL stands for **Read-Eval-Print Loop**. It's an interactive environment where:

- **Read** — Python reads one line of your code

- **Eval** — Python evaluates (executes) that line
- **Print** — Python prints the result
- **Loop** — Python loops back and waits for the next line

The REPL is perfect for quick experiments, testing small ideas, or learning how Python works.

Real-World Applications

- IT administrators use Python scripts to automate server monitoring
- DevOps engineers write Python to manage cloud infrastructure
- System administrators use Python to batch-rename files, parse logs, and generate reports
- The REPL is used daily by developers to quickly test snippets before adding them to larger programs

---

□ Lab Overview

Objectives	Use the REPL; create and run a `.py` script
Estimated Time	30–45 minutes
Prerequisites	Python 3.8+ installed, terminal access

Part A — Using the Python Interactive Shell (REPL)

Step 1: Launch the REPL

Open your terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type:

```
python3
```

> **Windows users:** You may need to type `python` instead of `python3`, depending on your installation.

You should see output similar to this:

```
Python 3.11.0 (default, Oct 24 2022, 18:03:06)
[GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

The `>>>` is the **REPL prompt** — Python is ready and waiting for your input.

Step 2: Type These Commands One at a Time

Type each line below and press **Enter** after each one. Observe what happens.

```
>>> print("Hello, World!")

Hello, World!

>>> print("My name is Python")

My name is Python

>>> 2 + 2

4

>>> exit()
```

> **Notice:** When you type an expression like `2 + 2` in the REPL, Python automatically prints the result. In a script file, you must use `print()` explicitly.

Step-by-Step Explanation

Command	What It Does
`print("Hello, World!")`	Calls the `print` function with the string `"Hello, World!"` as input
`print("My name is Python")`	Same function, different string argument
`2 + 2`	Python evaluates the expression and the REPL displays the result `4`
`exit()`	Closes the REPL and returns to the regular terminal

Part B — Writing Your First Script File

A **script** is a `.py` file containing Python code that runs all at once from top to bottom. This is how real programs are built.

Step 1: Create the File

Open your code editor, create a new file called `hello.py`, and type the following **exactly** (do not copy-paste):

```
# My first Python script
# The hash symbol starts a comment — Python ignores it

print("Hello, World!")
print("Welcome to Python programming!")
print("I am learning IT automation.")
```

Step 2: Run the Script

Save the file. In your terminal, navigate to the folder where you saved it, then run:

```
python3 hello.py
```

Expected Output

```
Hello, World!
Welcome to Python programming!
I am learning IT automation.
```

Code Execution — What Happens?

When you run `python3 hello.py`, Python:

1. Opens the file and reads it from top to bottom
2. Encounters the # comment lines — ignores them completely
3. Reaches `print("Hello, World!")` — calls the print function, which outputs the string to the screen
4. Continues to the next `print()` call, and the next
5. Reaches the end of the file — program terminates

Step-by-Step Explanation

```
# My first Python script
```

This is a **comment**. Python completely ignores anything after # on the same line. Comments are notes for humans reading the code.

```
print("Hello, World!")
```

`print` is a **built-in function** — it's included with Python automatically. The text inside the parentheses and quotes is called a **string argument**. Python displays it on screen and adds a newline at the end.

```
print("Welcome to Python programming!")
print("I am learning IT automation.")
```

Each `print()` call outputs its text on a new line.

□ Key Concepts

Concept	Definition
Interpreted language	Code is executed line by line at runtime, not pre-compiled
REPL	Interactive shell — Read, Eval, Print, Loop
Script	A `.py` file containing Python code run as a complete program
`print()` function	Built-in function that displays output to the screen

Concept	Definition
Comment	A line starting with # that Python ignores — used for human notes
String	Text wrapped in quotes: `"Hello"` or `'Hello'`

⚠ Common Mistakes

1. Missing Quotes Around Strings

```
# ▫ WRONG — Python treats Hello as a variable name (which doesn't exist)
print(Hello)

# ▫ CORRECT
print("Hello")
```

2. Wrong Capitalization of `print`

```
# ▫ WRONG — Python is case-sensitive
Print("Hello")
PRINT("Hello")

# ▫ CORRECT — always lowercase
print("Hello")
```

3. Mismatched Quotes

```
# ▫ WRONG — opened with double quote, closed with single quote
print("Hello, World!')

# ▫ CORRECT — matching quotes
print("Hello, World!")
print('Hello, World!')   # single quotes also work
```

4. Forgetting to Save the File

You edit `hello.py`, run it, and the old output appears. Always save before running. VS Code shows a dot • on the tab when there are unsaved changes.

5. Wrong Directory in Terminal

```
# ▫ If your file is in ~/Documents/python/ but you're in ~
python3 hello.py   # Error: no such file

# ▫ Navigate first
cd ~/Documents/python/
python3 hello.py
```

🏋️ Additional Practice

Write a script called `about_me.py` that prints:

- Your name
- Your current goal (e.g., "I want to automate IT tasks")
- Three things you hope to learn from Python

Use at least one comment explaining what the script does.

□ Interview Tips

Q: What is the difference between an interpreted and compiled language?

> Interpreted languages (like Python) execute code line by line at runtime. Compiled languages (like C++) translate all code into machine code before running. Interpreted languages are slower but more flexible and easier to develop with quickly.

Q: What does REPL stand for and what is it used for?

> REPL stands for Read-Eval-Print Loop. It's an interactive shell for running Python one line at a time — great for testing small ideas or exploring how Python behaves.

Q: What is a `.py` file?

> A `.py` file is a Python script — a plain text file containing Python code that is executed by the Python interpreter.

🌍 Real-World Applications

Scenario	How This Applies
Server monitoring scripts	IT admins write `.py` scripts that run on a schedule to check server health
Log parsing automation	Python scripts read log files and print summaries automatically
Quick data validation	Use the REPL to quickly test a formula or logic before adding it to a larger script
Onboarding automation	Scripts that print welcome messages, setup instructions, or config info to new users
Deployment scripts	DevOps pipelines run `.py` files to print status updates during software deployment

---

🏆 Pro Tips

- **Use the REPL constantly.** Any time you're unsure how something works, open the REPL and test it in 10 seconds.
- **Name your files descriptively.** `hello.py` is fine for learning, but real scripts should have meaningful names like `check_disk_usage.py`.
- **One concept per file when learning.** Keep your practice scripts small and focused — it's easier to debug and review later.
- **Comments are not optional.** Even in tiny scripts, get in the habit of writing what your code does. Your future self will thank you.

□ Mini Revision

Quick self-check — answer these from memory:

- ☐ What command launches the Python REPL?
- ☐ What command exits the REPL?
- ☐ What symbol starts a comment in Python?
- ☐ What is the difference between running code in the REPL vs. a script file?
- ☐ What does `print("Hello")` do?

□ Cross References

Previous Topic	— (This is the beginning!)
Next Topic	Lab 1.2 — Understanding print()
Related Concepts	Variables (Week 2), Functions (Week 4)
Week README	Week 1 Overview

← *Week 1 README* | *Next: Lab 1.2 — Understanding print()* →